

Object Detection of Pornography Animated Images based on Improved YOLOv5

Xichao Wang

XichaoWang97@outlook.com | 2445879288@qq.com

Abstract

Due to the differences between animated pornography content and real pornography imagery, many algorithms designed for general pornography image detection underperform when applied to animated content. To address this issue, we propose a novel method for processing animated pornography images, which integrates YOLOv5 with RepLKnet. This approach utilizes deep learning and object detection to identify sensitive areas within animated pornography images, allowing for shielding operations such as pixelate. Experimental results show as follows: on the self-built dataset, our best proposed algorithm has the mean Average Precision(mAP) reached 46.386%. Compared to YOLOv5s, the proposed algorithm has the mAP of detection increased by 6.329 percentage points, and increased by 0.430 percentage points compared to YOLOv5x. Code is available at <https://github.com/XichaoWang97/Object-Detection-of-Pornography-Animated-Images>

1. Introduction

With the advancement of the internet, significant researches have been conducted over the past decade on filtering pornography images online. Traditional methods of pornography detection rely on skin color detection, which involves assessing whether an image contains human nudity [1], using color characteristics as the identifying feature for pornography content. In the selection of color feature, the RGB color space is not stable and is significantly affected by factors such as lighting. Therefore, the YCrCb and HSV color spaces are commonly employed for detection, and texture features are additionally utilized to aid in the detection process.

Over the past decade, with the advancement of deep learning, Mohamed N. Moustafa [2] utilized Convolutional Neural Network (CNN) for pornography detection. Not long after, Kai Li and Junliang Xing et al. [3] posited that low-level features such as skin color and texture are crucial for the identification of pornographic images, necessitating increased weight training. This underscores the current de-

velopment in pornography detection, which has evolved to integrate deep learning with traditional skin color detection methods. Consequently, the detection of pornographic images has primarily been approached as an image classification problem. In 2016, Xinyu Ou and He-fei Ling et al. [4] applied object detection method to the identification of pornography content in images, marking the regions containing nudity with bounding boxes.

After considered about the above ideas, we intend to employ deep learning techniques for sensitive region detection in images, this will allow us to perform precise operations within the target detection bounding box, to effectively shield sensitive areas in pornographic images.

Certainly, a more comprehensive approach involves the use of skin color detection methods within the bounding box. This way need to extract the range of skin color value in non-sensitive areas, then calculating the skin exposure rate in sensitive areas, and controlling whether to employ a shielding operation by setting a threshold.

2. Related works

2.1. YOLOv5

Redmon et al. proposed the YOLO algorithm in 2016 [5], and then continuously improved it, releasing YOLOv5 in June 2020.

The YOLOv5 framework consists of four parts: Input, Backbone, Neck, and Head. The input part of YOLOv5 includes Mosaic augmentation, Random Affine transformation, Mixup augmentation, HSV color space enhancement, etc. The backbone part consists of CBS (Conv2d + Batch Normalization [6] + SiLU activation function [7]) convolution modules, C3 structure, and SPPF modules [8]. The neck part adopts the FPN [9] + PAN [10] feature pyramid structure that combines top-down and bottom-up multi-scale fusion. The head part generates predicted bounding boxes and classes by three detection heads of different sizes, uses CIOU [11] loss function to regress and correct the targets, and finally eliminates redundant boxes through NMS [12] to improve the recognition accuracy of the targets.

2.2. RepLKNNet

RepLKNNet [13] is a novel convolutional neural network (CNN) architecture that uses large kernels of size 31×31 , instead of the commonly used 3×3 kernels. It is designed to achieve high performance and efficiency for computer vision tasks, such as image classification, segmentation, and detection. RepLKNNet is inspired by the advances in vision transformers [14] (ViTs), which use large self-attention modules to capture long-range dependencies, it aims to mimic the benefits of ViTs using pure convolution operations, which are more hardware-friendly and easier to optimize.

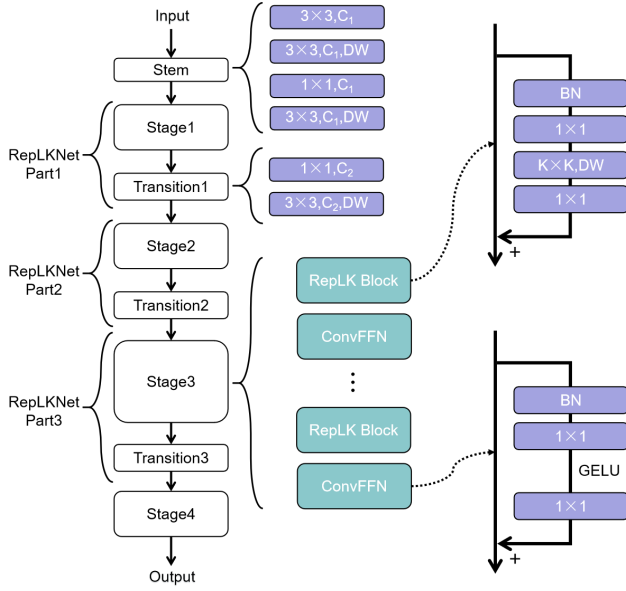


Figure 1. Network of RepLKNNet, for availability, we divide the first three Stage Blocks and Transition Blocks into three parts.

RepLKNNet consists of Stem, Stages and Transitions. Stem is the beginning layers to capture more details by several conv layers at the beginning. It contains a 3×3 with $2 \times$ downsampling, a DW (Depthwise separable convolution) [15] 3×3 layer, a 1×1 conv, and another DW 3×3 layer for downsampling. Stages contains several RepLK Blocks and ConvFFN Blocks. RepLK Blocks contains a BN [6] layer, 1×1 conv, DW conv and another 1×1 conv. (1×1 layers are used to increase the depth). ConvFFN Block contains a BN layer, two 1×1 layers and GELU [16]. Transition Blocks are placed between stages, which first increase the channel dimension via 1×1 conv and then conduct $2 \times$ downsampling with DW 3×3 conv. Each stage has three architectural hyper-parameters: the number of RepLK Blocks B , the channel dimension C , and the kernel size K . RepLKNNet architecture is defined by $[B1,$

$B2, B3, B4], [C1, C2, C3, C4], [K1, K2, K3, K4]$.

3. Object Detection Network

3.1. Structure of the network

This object detection task is applied to animated images, and the targets in this object detection are various parts of the human body, which often exhibit certain spatial relationships. For these reasons, we decided to select the RepLKNNet architecture for its large convolution kernels. We hope that these larger kernels will afford a large receptive fields, allowing the model to learn the positional correlation features between different detection targets, thus improving the accuracy of target detection.

To start, we use RepLKNNet to replace the backbone of YOLOv5 directly, and with kernel sizes $K=[31, 29, 27, 13]$, $B=[2, 2, 18, 2]$, $C=[128, 256, 512, 1024]$ in the four stages. Structure is shown in Fig. 2.

3.2. Schemes to make the network lightweight

After replacing the backbone with RepLKNNet, the model's parameter count significantly exceeded. Consequently, we implemented several strategies to make the network more lightweight. We experimented with connect the backbone with the head of YOLO directly (Fig. 3); removing Stage4 and connect Part3 with the first Conv layer of the Neck (Fig. 4); adjusting the third parameter of B , which corresponds to the number of RepLK Blocks and ConvFFN in Part3.

4. Experiment

4.1. Dataset

Datasets are an indispensable foundational requirement for experiments. Generally, due to copyright reasons or regulatory policies, currently available open-source pornographic datasets are extremely scarce, and datasets for animated pornography are even harder to find. Due to the sensitive content and related policies, such datasets are difficult to open source. To address this issue, this paper has created a dataset concerning animated pornography image.

We use labeling to build our animated dataset, and call it ACG dataset, which contains a total of 1000 images. It consists of 500 non-pornographic images and 500 pornographic images. The images are all crawled from the Pixiv. Among these 1000 images, we use 900 images as the training set and 100 images as the validation set.

Our ACG dataset contains 8 labels so far, includes character (to detect the location of animated characters), head, eye, hand, breast, hip, private parts of male and female. In the 8 labels, breast, hip and private parts are all sensitive labels, our goal is to get the accurate bounding box of them.

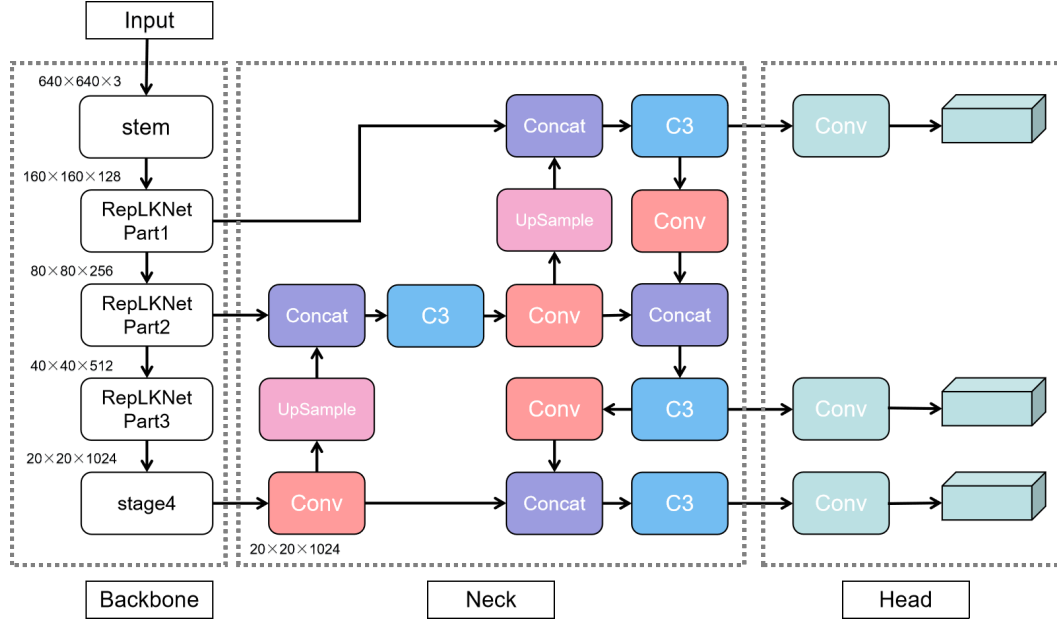


Figure 2. Combination of YOLOv5 and RepLKNet, it is also the initial scheme

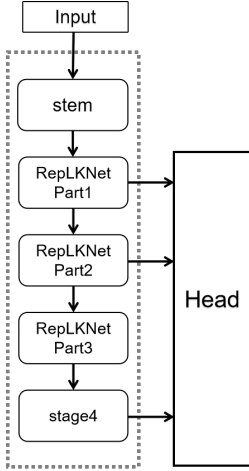


Figure 3. Connect the backbone with the head of YOLO directly

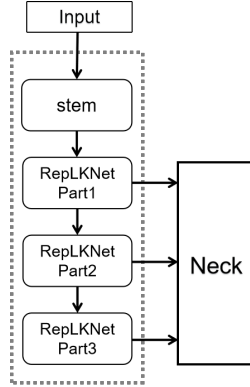


Figure 4. Network structure without Stage4

Training Parameters	Values
momentum	0.937
weight_decay	0.0005
lr0	0.01
Lrf	0.01
batchsize	15
epochs	300

Table 1. Settings of Partial Hyperparameter

after light-weighting are shown in Table 2. This table provides four indicators: precision, recall, mAP_0.5, and mAP_0.5:0.95. Here are all abbreviations of all sorts of networks and explanations in this experiment:

- Rep: the initial scheme
- RepLarge: Rep with Depth=1.33, Width=1.25
- WithoutNeck: connect to the Head directly
- WithoutStage4: scheme without stage4
- B=[2,2,8,2]: Rep with different B value
- B=[2,2,8,2]Large: Depth=1.33, Width=1.25

4.2. Experiment and Results

The experimental environment is described as follows: The operating system is Windows 10, with the deep learning framework PyTorch 2.2.0, utilizing a GPU RTX 4090 with 24 GB of Video RAM for model training. The Python version is 3.11, and the CUDA version is 11.0. Some of the hyperparameters set for the network during the training process are shown in Table 1.

Network Training: The comparison results of the original combination network and the improved networks

Observations from Table 2 indicate that after replacing the backbone network of YOLOv5s with RepLKNet, all indicators significantly improved. However, connecting RepLKNet with the Head of YOLOv5 (without Neck) can not enhance the performance of network; on the contrary, mAP_0.5 decreased by 0.47 percentage points, and mAP_0.5:0.95 decreased by 0.72 percentage points. In

other light-weighting schemes, removing stage4 improved the indicators compared to YOLOv5s, but not as much as the initial direct backbone replacement. Through analysis, we believe that the direct connection to the head, due to the lack of the concat and upsample layers, resulted in no feature fusion and enhancement, thus leading to lower performance across all indicators.

The above failures did not completely negate our approach to making the network lighter. In the end, during the tuning process of B, we discovered some schemes that outperformed the direct backbone replacement, especially when $B=[2,2,8,2]$, which showed the best performance.

	Depth	Width	Params(M)
YOLOv5s	0.33	0.50	7.04
YOLOv5x	1.33	1.25	86.26
Rep	0.33	0.50	90.23
RepLarge	1.33	1.25	223.35
$B=[2,2,8,2]$	0.33	0.50	60.04
$B=[2,2,8,2]$ Large	1.33	1.25	193.16

Table 2. The params the network with different Depth (Depth_multiple) and Width (Width_multiple), Depth controls the number of some layers, Width controls the channels of the whole network.

Results could be found in Table 3. Compared with YOLOv5x, our initial scheme is larger than its number of parameters. By slimming our network, the problem of excessive parameters is alleviated to some extent. Later, we verified on our dataset that the indicators of YOLOv5x are better than the previous schemes. However, by adjusting the hyperparameters Depth_multiple and Width_multiple, the new original scheme and $B=[2,2,8,2]$ scheme show slightly better indicators than yolov5x, but the increase in the number of parameters is still huge. Comparison of detection results with some different models are shown in Fig. 5 and Fig. 6.

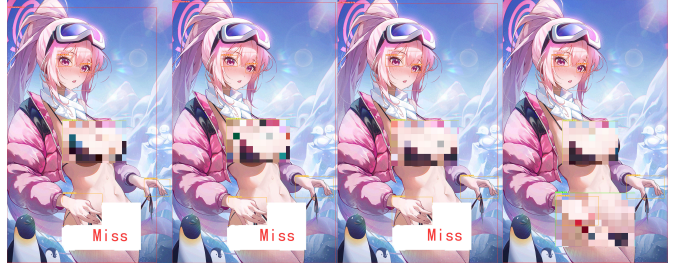


Figure 5

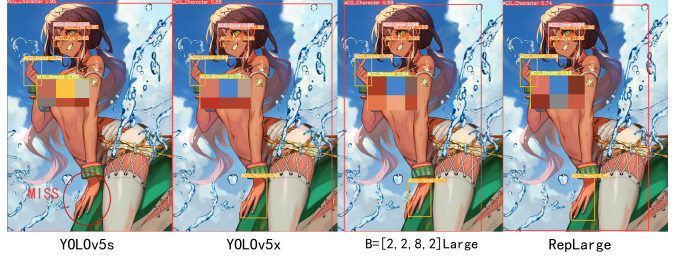


Figure 6

5. Conclusion and Future Work

This paper proposes an algorithm based on an improved YOLOv5 model. By integrating RepLkNet with YOLOv5, the algorithm detects sensitive areas of the human body through object detection. Experimental results show that the algorithm achieves good detection accuracy.

Future work:

- 1) We will continue to explore how to reduce the number of algorithm parameters and increase the detection rate.
- 2) In our self-build dataset, instances of sensitive areas are even rarer. Therefore, it is necessary to expand the dataset. Moreover, a comprehensive animated pornography dataset needs a richer variety of art styles.
- 3) Introduce skin color-related algorithms to calculate the skin exposure rate within the target area. By setting a reasonable threshold, we aim to achieve automatic cen-

	precision	recall	mAP_0.5	mAP_0.5:0.95
WithoutNeck	0.87011	0.60514	0.68583	0.39331
YOLOv5s	0.8256	0.62514	0.69059	0.40057
WithoutStage4	0.85162	0.64021	0.70966	0.43033
Rep	0.89363	0.62333	0.70232	0.43122
$B=[2,2,8,2]$	0.88668	0.63083	0.70262	0.44382
YOLOv5x	0.88664	0.65381	0.71158	0.45956
$B=[2,2,8,2]$ Large	0.90037	0.64877	0.71232	0.46312
RepLarge	0.8668	0.65523	0.72753	0.46386

Table 3. Results of different detection networks

sorship due to excessive exposure rates.

References

- [1] Margaret M Fleck, David A Forsyth, and Chris Bregler. Finding naked people. In *Computer Vision—ECCV’96: 4th European Conference on Computer Vision Cambridge, UK, April 15–18, 1996 Proceedings Volume II 4*, pages 593–602. Springer, 1996. 1
- [2] Mohamed Moustafa. Applying deep learning to classify pornographic images and videos. *arXiv preprint arXiv:1511.08899*, 2015. 1
- [3] Kai Li, Junliang Xing, Bing Li, and Weiming Hu. Bootstrapping deep feature hierarchy for pornographic image recognition. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 4423–4427. IEEE, 2016. 1
- [4] Xinyu Ou, Hefei Ling, Han Yu, Ping Li, Fuhao Zou, and Si Liu. Adult image and video recognition by a deep multi-context network and fine-to-coarse strategy. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(5):1–25, 2017. 1
- [5] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016. 1
- [6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015. 1, 2
- [7] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018. 1
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE transactions on pattern analysis and machine intelligence*, 37(9):1904–1916, 2015. 1
- [9] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2117–2125, 2017. 1
- [10] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. Path aggregation network for instance segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8759–8768, 2018. 1
- [11] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 12993–13000, 2020. 1
- [12] Alexander Neubeck and Luc Van Gool. Efficient non-maximum suppression. In *18th international conference on pattern recognition (ICPR’06)*, volume 3, pages 850–855. IEEE, 2006. 1
- [13] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Scaling up your kernels to 31x31: Revisiting large kernel design in cnns. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11963–11975, 2022. 2
- [14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 2
- [15] C Fran et al. Deep learning with depth wise separable convolutions. In *IEEE conference on computer vision and pattern recognition (CVPR)*, 2017. 2
- [16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016. 2